A

MINI PROJECT REPORT ON

Hospital Emergency Management System

SUBMITTED TO SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY (B.TECH)

(Artificial Intelligence & Data Science)

BY

Siddhesh M Bombale	**32263.**
Dnyaneshwari S Kadam	**32267.**
Vishal B Karajagi	**32269.**
Ayushi N Patil	**32273.**

Under The Guidance Of

Mrs. MAYURI FEGADE

Dr. D. Y. Patil College of Engineering and Innovation, Varale

Department of Artificial Intelligence - Data Science and Machine Learning

Dr. D.Y. Patil College of Engineering & Innovation
Varale – Talegaon, Pune – 410507

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

Academic Year: 2026 – 2027

Dr. D. Y. Patil College of Engineering and Innovation, Varale

Department of Artificial Intelligence - Data Science and Machine Learning

CERTIFICATE

This is to certify that the Mini Project entitled

"Hospital Emergency Management System"

Siddhesh M Bombale	32263.
Dnyaneshwari S Kadam	32267.
Vishal B Karajagi	32269.
Ayushi N Patil	32273.

is a bonafide work carried out by them under the guidance of **Mrs. Mayuri Fegade**, in partial fulfillment of the requirements of Savitribai Phule Pune University, Pune for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science.

Mrs. Mayuri Fegade
Guide

Dr. Dipannita Mondal
Head- AI- DS & ML

Dr. Suresh Mali
Director

Signature of Internal Examiner

Signature of External Examiner

ACKNOWLEDGENT

It is our pleasure to express our sincere gratitude to all those who helped us complete this mini project successfully. We are highly indebted to our project guide**, Mrs. Mayuri Fegade**, for her valuable guidance, continuous supervision, and support throughout the development of the Hospital Emergency Management System.

We would also like to express our gratitude to the Head of the Artificial Intelligence – Data Science and Machine Learning Department**, Dr. Dipannita Mondal**, for providing encouragement and the necessary facilities. We are thankful to all our teachers and friends for their valuable suggestions and cooperation.

Finally, we would like to thank everyone who directly or indirectly helped us complete this mini project successfully.

Name of the Student and Sign

Place:

Date:

Contents

1	**Chapter 1: Introduction**	**5**
2	**Chapter 2: Problem Statement**	6
3	**Chapter 3: Objectives**	8
4	**Chapter 4: Motivation**	9
5	**Chapter 5: Scope and Rationale of the Project**	11
6	**Chapter 6: Methodological Details**	12
7	**Chapter 7: Results**	14
8	**Chapter 8: Analysis**	17
9	**Chapter 9: Inferences**	18
10	**Chapter 10: Conclusion**	19
11	**Chapter 11: Suggestions**	20
12	**Chapter 12: Acknowledgments**	21
13	**Chapter 13: List of References**	22

Chapter 1: Introduction

The purpose of this mini project is to develop a Hospital Emergency Management System that organizes patients according to medical priority. The system uses a priority queue for emergency cases and a linked list to store patient records dynamically. Through this project, we will:

- Register patients and store their basic details dynamically.
- Assign priority levels according to the urgency of the patient's condition.
- Treat emergency patients before normal cases.
- Complete treatment, remove treated patients from the queue, and maintain updated patient records.

Background: Hospitals receive patients with different medical conditions. During busy hours, handling every patient only by arrival time may delay urgent cases. A priority-based system helps staff identify the patients who need attention first.

System Context: Emergency patients such as accident or critical-care cases must be considered before routine cases. Priority assignment should be clear and consistent.

Technical Primer: A priority queue selects patients according to their assigned priority. A linked list stores patient nodes dynamically and can grow or shrink as patients are registered or treatment is completed.

Expected Impact: The project demonstrates how basic data structures can support better queue organization, reduce confusion at reception, and maintain patient records in a simple computer-based system**.**

2: Problem Statement

In a busy hospital, manual patient handling can create delays and confusion, especially when emergency and normal cases arrive together. A simple first-come-first-served process may not always serve the most urgent patient first.

- Patient details may be written in separate registers and become difficult to update.
- Emergency cases may wait behind normal cases if priority is not considered.
- Searching, updating, and removing patient records manually can take extra time.

This project aims to:

1. Develop a system to register patients and store their details.
2. Assign emergency or normal priority to each registered patient.
3. Process patients according to priority while maintaining FIFO order among patients with the same priority.

- **Operational Requirements:**
    - Patient Registration: Store a patient ID, name, age, symptoms, and assigned priority.
    - Priority Handling: Emergency cases are selected before normal cases; equal-priority cases follow arrival order.

- **Dynamic Records**: Use a linked list so patient nodes can be added or removed when needed.
- **Record Maintenance:** Update treatment status and remove completed cases without disturbing other waiting patients.

Chapter 3: Objectives

1. Develop the Patient Queue:

- Register new patients with unique IDs and basic details.
- Place each patient in the correct priority position in the waiting queue.

2. Assign and Manage Priority:

- Give emergency cases higher priority than normal cases.
- Maintain FIFO order among patients having the same priority.

3. Support Treatment Completion:

- Display the next patient who should receive treatment.
- Mark treatment as completed and remove or archive the patient record.

4. Dynamic Record Management:

- Use linked-list nodes to add, search, update, and delete patient records.
- Avoid a fixed-size array so the number of records can change at runtime.

5. Simple User Interaction:

- Provide a menu for registration, viewing the queue, assigning priority, and completing treatment.
- Display clear messages when a patient is not found or the queue is empty.

Chapter 4: Motivation

The motivation for this project comes from the need to organize patient flow clearly and give urgent cases timely attention in a hospital environment:

1. **Patient Safety:**
    - Emergency cases may require immediate assessment and should be identified quickly.
    - A patient with a serious injury should not be kept behind routine cases only because they arrived later.
    - A clear queue can help reception and medical staff understand which patient is waiting next.
    - Keeping records in one system also makes it easier to check patient details and treatment status.
2. **Data Structure Learning:**
    - The project gives practical experience with queues, priority handling, linked lists, and node operations.
    - It shows how data structures can solve a real-world organization problem.
3. **Need for Accuracy:**
    - Correct patient identification and careful priority assignment are important for safe workflow.
    - This project connects classroom concepts with a useful hospital scenario while keeping the implementation simple and understandable.

4. System Architecture:

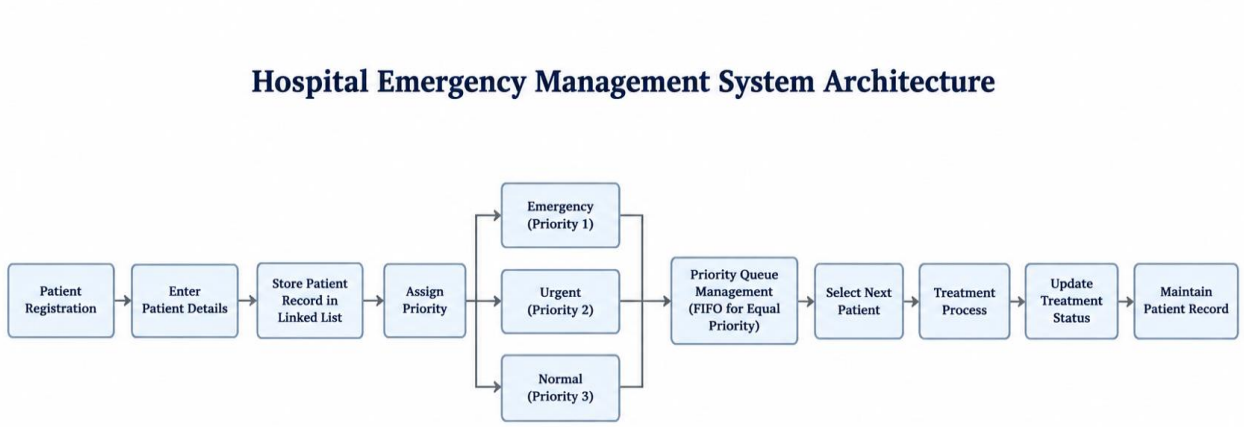


Image Description:

This diagram description represents the flow of the Hospital Emergency Management System. A patient is registered and their details are stored in a linked list. The system assigns a priority and places the patient in the waiting queue. The next patient is selected according to priority, with FIFO order for equal-priority cases. After treatment, the record is marked complete and the queue is updated.

Chapter 5: Scope and Rationale of the Project

Scope:

- **Technical Boundaries:**
    - Focuses on patient registration, priority-based queue management, and linked-list record storage.
    - Supports emergency and normal priority levels, FIFO ordering within the same level, treatment completion, and record viewing.
- **Practical Constraints:**
    - The project is a mini-project prototype using sample patient details and does not replace a hospital information system.
    - It does not diagnose medical conditions or automatically decide clinical urgency.

Rationale:

1. **Hands-On Learning:**
    - Students can practice insertion, searching, priority selection, deletion, and traversal using linked lists and queues.
2. **Simple and Flexible Storage:**
    - Dynamic nodes allow patient records to be added and removed without declaring a fixed record capacity in advance.
3. **Real-World Relevance:**
    - A priority queue models the need to handle urgent cases before routine cases while preserving arrival order for equal priorities.

Chapter 6: Methodological Details

The project is organized into the following development phases:

1. Patient Registration

- Technologies Used: C/C++ or Java for the program logic, with a console menu or a simple graphical interface.
- **Features:**
    - Enter patient ID, name, age, and symptoms.
    - Select the patient priority as emergency or normal.
    - Create a linked-list node and add it to the patient records.

2. Priority Queue Management

- Store waiting patients in priority order.
- **Queue Rules:**
    - Emergency patients are placed ahead of normal patients.
    - Patients with equal priority are arranged by arrival time.
    - Normal cases follow FIFO order unless a higher-priority emergency case is waiting.

3. Linked-List Record Operations

- **Patient Record Storage:**
    - Each node contains patient details, priority, treatment status, and a link to the next node.

- Operations:
    1. Traverse the list to display all registered patient records.
    2. Search for a patient by ID and display their details.
    3. Update priority or status when authorized by the operator.
- **Remove or archive a record after treatment completion.**

- **Treatment Completion:**
    - Select the highest-priority patient at the front of the waiting queue.
    - Steps:
        1. Display the patient selected for treatment.
        2. Mark the patient as treated and update the waiting queue.
    - **Show the next eligible patient, if any.**
- **Validation:**
    - Check for duplicate IDs, invalid priority values, and missing patient records.
    - **Display a suitable message when no patients are waiting.**

4. Testing and Record Maintenance

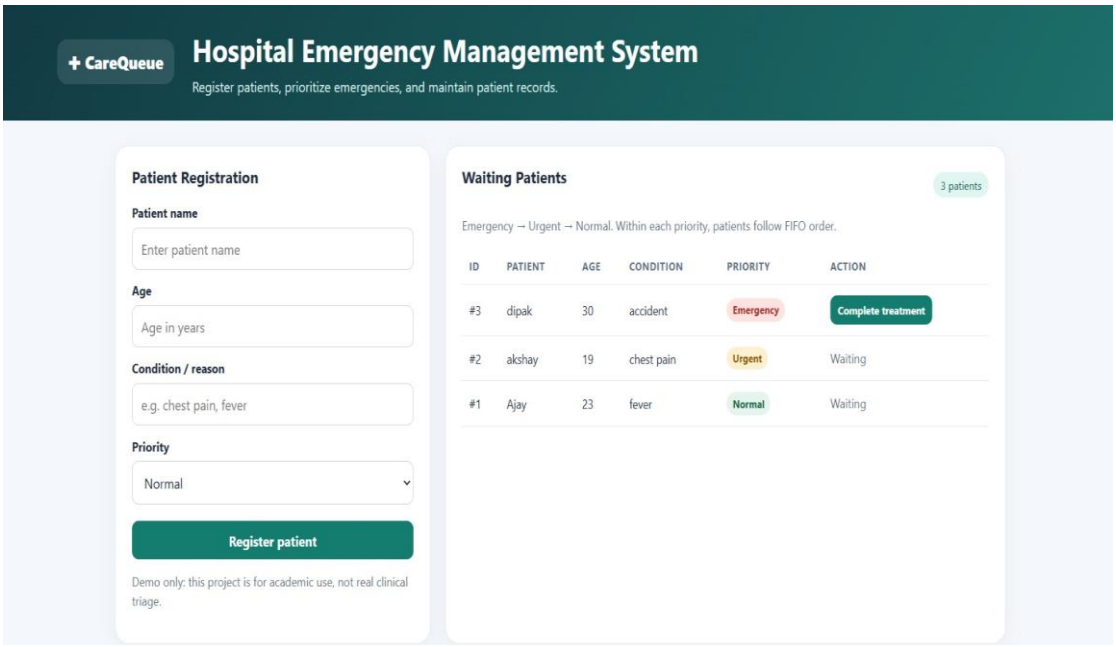
- **Test Cases:**
    1. Register emergency and normal patients in different arrival orders.
    2. Verify that emergency patients are selected first and equal-priority cases remain FIFO.
    3. Confirm that completed patients no longer appear in the waiting queue.
- **Testing:**
    - Check registration, search, priority assignment, display, and completion operations.
    - Use sample data to verify queue order and linked-list updates.

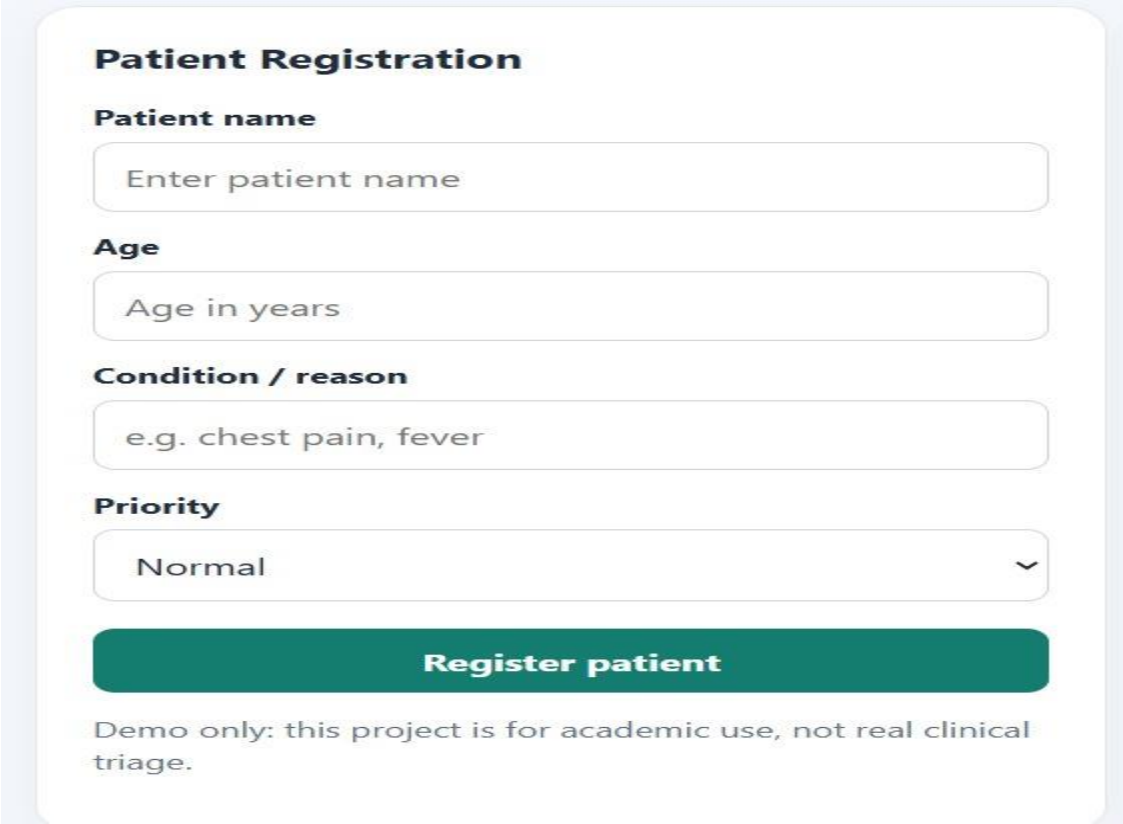
Chapter 7: Results

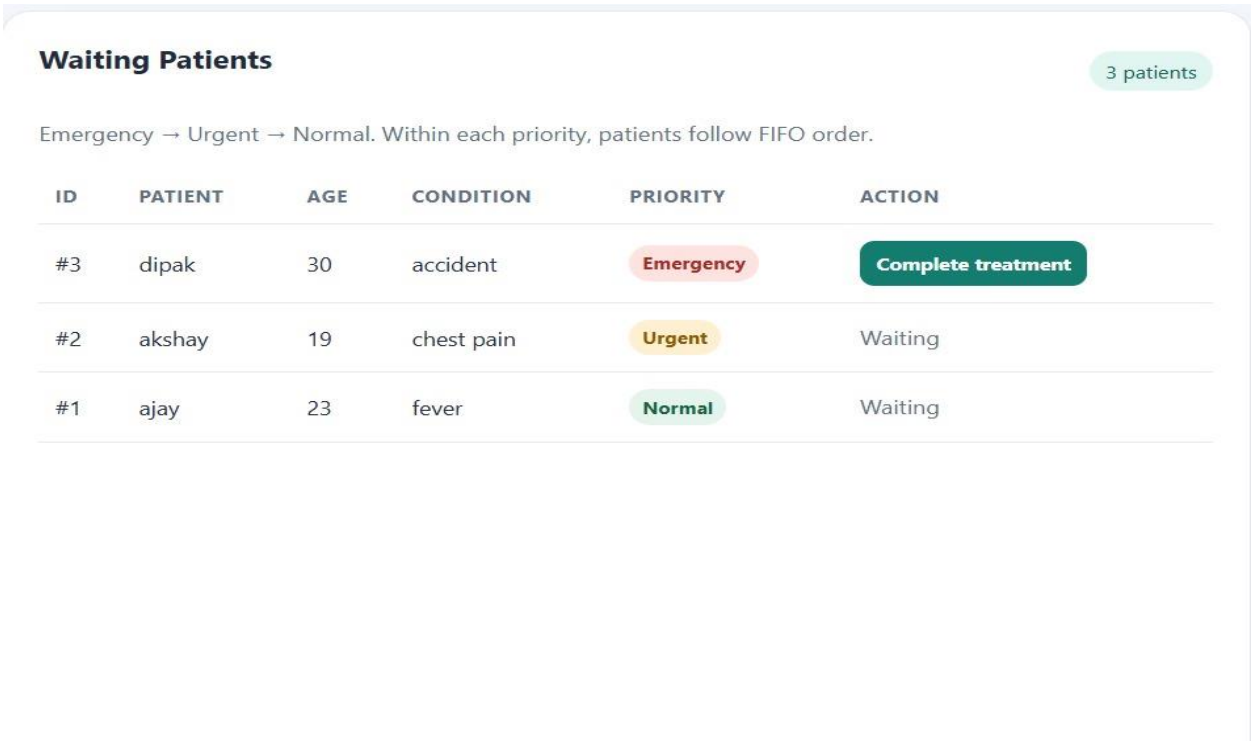
Aspect	Expected System Behaviour	Verification
Patient Registration	Patient details stored in a linked-list node	New record appears in the record list
Priority Handling	Emergency patients selected before normal cases	Emergency case moves ahead of normal cases
FIFO Ordering	Equal-priority patients follow arrival order	Earlier arrival is selected first
Treatment Completion	Treated patient removed from waiting queue	Completed case no longer waits
Record Search	Patient can be found using a unique ID	Correct details are displayed
Dynamic Storage	Records can be added and removed at runtime	List links remain valid after updates
Empty Queue	System handles no waiting patients safely	Clear message is displayed

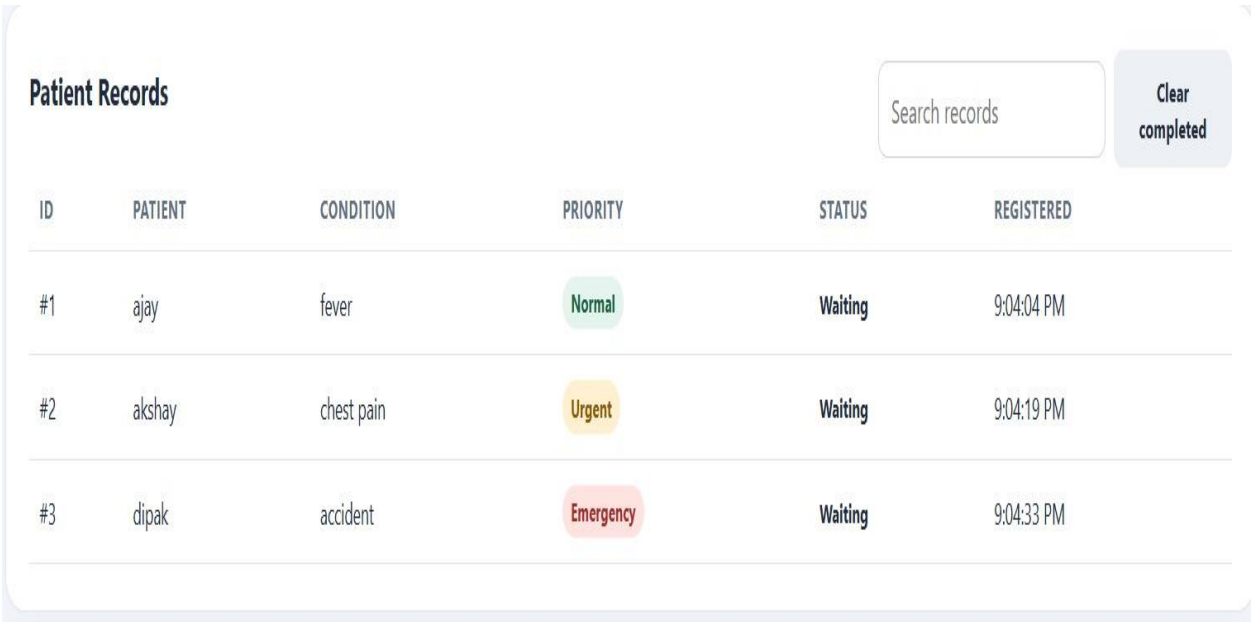
Key Observations:

- The priority queue selects emergency patients before normal cases, even when a normal patient arrived earlier.
- Patients with the same priority are handled in their arrival order, and completed cases are removed from the active waiting queue.


+ CareQueue
Hospital Emergency Management System
Register patients, prioritize emergencies, and maintain patient records.
Patient Registration
Patient name
Enter patient name
Age
Age in years
Condition / reason
e.g. chest pain, fever
Priority
Normal
Register patient
Demo only: this project is for academic use, not real clinical triage.
Waiting Patients
3 patients
Emergency → Urgent → Normal. Within each priority, patients follow FIFO order.
ID PATIENT AGE CONDITION PRIORITY ACTION
#3 dipak 30 accident Emergency Complete treatment
#2 akshay 19 chest pain Urgent Waiting
#1 Ajay 23 fever Normal Waiting


Patient Registration
Patient name
Enter patient name
Age
Age in years
Condition / reason
e.g. chest pain, fever
Priority
Normal
Register patient
Demo only: this project is for academic use, not real clinical triage.


Waiting Patients
3 patients
Emergency → Urgent → Normal. Within each priority, patients follow FIFO order.
ID
PATIENT
AGE
CONDITION
PRIORITY
ACTION
#3
dipak
30
accident
Emergency
Complete treatment
#2
akshay
19
chest pain
Urgent
Waiting
#1
ajay
23
fever
Normal
Waiting


Patient Records
Search records
Clear completed
ID
PATIENT
CONDITION
PRIORITY
STATUS
REGISTERED
#1
ajay
fever
Normal
Waiting
9:04:04 PM
#2
akshay
chest pain
Urgent
Waiting
9:04:19 PM
#3
dipak
accident
Emergency
Waiting
9:04:33 PM

Chapter 8: Analysis

Key Findings:

- **Priority Queue**: It helps the system select the most urgent waiting patient before routine cases**.**

- **FIFO Rule:** Patients assigned the same priority retain their arrival order.
    - Linked List: Dynamic nodes support insertion and deletion as patient records change.
    - Record Maintenance: Searching and updating records helps keep the displayed information current.

- **Usability:** A menu-driven interface makes common actions easy to understand and perform**.**

- **Functional Checks:**
    - Queue tests should confirm that emergency cases are selected first and equal-priority cases preserve FIFO order.
    - Linked-list tests should confirm that registration, searching, and removal update the record chain correctly**.**

- **Limitations:**
    - The system depends on staff entering accurate patient details and assigning an appropriate priority.
    - The prototype does not provide medical diagnosis, real-time monitoring, or secure integration with hospital databases.

Chapter 9: Inferences

- **Data Structure Selection:**
  A priority queue is suitable for choosing patients by urgency, while a linked list is useful for dynamic patient record storage.
  Combining both structures helps separate waiting order from the full set of maintained records.

- **Workflow Improvement:**
  A clearly displayed queue can help staff identify the next patient and reduce confusion at reception.

- **Testing Inference:**
  Sample cases can verify priority ordering, FIFO behavior, and correct updates after treatment completion.

- Priority must be assigned responsibly by qualified hospital staff.

- The system is an educational prototype and should not independently make clinical decisions.

Chapter 10: Conclusion

This project demonstrates:

- How patients can be registered and stored as dynamic linked-list records.
- How a priority queue can handle emergency cases before normal cases.
- How FIFO order is maintained for patients with the same priority and records are updated after treatment.

Critical Takeaway: Combining a priority queue and linked list provides a simple way to organize waiting patients and maintain changing records.

- **Future Enhancements:**
  - Add a graphical interface with patient search, queue display, and treatment status.
  - Connect the application to a database for persistent records and controlled access.
- **Include timestamps, audit logs, and staff-managed priority updates for improved traceability.**

Chapter 11: Suggestions

1. **For Developers:**
    - Validate patient IDs and required fields before registration.
    - Keep priority ordering and FIFO behavior consistent.
    - Test insertion, search, update, and deletion for different queue states.
    - Handle empty queues and missing patient IDs with clear messages.
    - Separate waiting-queue operations from permanent record maintenance.
2. **For Hospital Staff:**
    - Enter patient information carefully and avoid duplicate records.
    - Assign emergency priority according to hospital procedures and clinical assessment.
    - Update treatment status as soon as the case is completed.
    - Review the waiting queue regularly to ensure that records are accurate.
3. **For Future Users:**
    - Protect patient information and restrict access to authorized staff.
    - Back up records if persistent storage is added.
    - Use a database and secure login for a production-level system.
    - Evaluate the prototype with realistic test cases before considering wider use.

Chapter 12: Acknowledgments

We sincerely thank **Mrs. Mayuri Fegade** for her guidance and support throughout this mini project. We are grateful to **Dr. Dipannita Mondal**, Head of the Artificial Intelligence - Data Science and Machine Learning Department, for encouragement and for providing the opportunity to work on this project.

We also thank our classmates and project team members for their cooperation in planning, implementing, and testing the Hospital Emergency Management System. Their suggestions helped us understand queue operations, linked-list record management, and the importance of checking different patient arrival scenarios.

Finally, we thank our teachers and all those who directly or indirectly supported the completion of this mini project.

Chapter 13: List of References

- Data Structures and Algorithms – Queue and Priority Queue concepts.https://flask.palletsprojects.com/
- Data Structures and Algorithms – Linked List operations and dynamic memory.https://www.openssl.org/docs/
- Oracle Java Documentation: https://docs.oracle.com/en/java/https://www.wireshark.org/docs/
- GeeksforGeeks: https://www.geeksforgeeks.org/data-structures/ https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- Programiz – Linked List and Queue tutorials: https://www.programiz.com/dsahttps://owasp.org/